

Pehlione DemoBank

Modern SEPA banking simulation with an atomic double-entry ledger



[Local UI](#) · [Gateway/API Health](#) · [Swagger](#) · [Jaeger](#) · [Grafana](#) · [Prometheus](#) · [Security Report](#)

[!IMPORTANT]

Demo-Banking – kein echtes Bankkonto. This project is a simulation. It does not connect to SEPA, PSD2, a bank, or a payment provider and never moves real money. It is not BaFin-approved or a production banking system.

Overview

Pehlione DemoBank is a full-stack banking sandbox for exploring German IBANs, SEPA-style payment orchestration and double-entry accounting. It combines a Go API, PostgreSQL ledger and responsive German Next.js interface in one Docker-ready repository.

Every new customer receives a fictional EUR Girokonto, a valid demo IBAN and a balanced **€500 opening credit** in one database transaction.

Highlights

Banking experience	Payments and ledger	Platform and security
Responsive German UI	Standard and instant transfers	HttpOnly session cookies
Collapsible desktop/mobile drawer	Scheduled payments	CSRF and CORS protection
IBAN reveal, copy and share	Standing orders	Endpoint rate limits
Account, transaction and profile views	Verification of Payee	Masked IBAN responses
Admin user/account controls	Idempotency keys	Strict JSON validation
SSE updates with polling fallback	Atomic debit and credit entries	Audit events and CodeQL

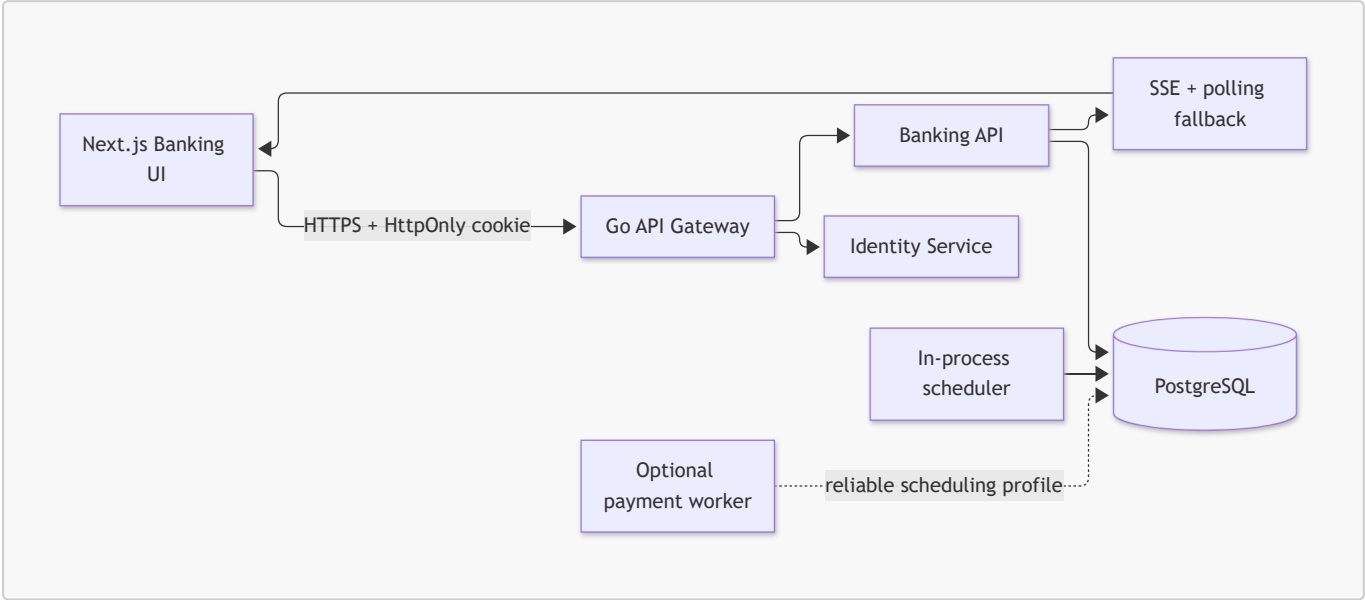
Supported payment flows

- **Umbuchung** — between a customer's own accounts.
- **Interne Überweisung** — between Pehlione demo customers.
- **SEPA-Überweisung** — simulated external EUR transfer.

- **SEPA-Echtzeitüberweisung** — simulated instant payment.
- **Terminüberweisung** — one-time scheduled payment.
- **Dauerauftrag** — recurring scheduled payment.

The payment state machine uses **DRAFT**, **AWAITING_CONFIRMATION**, **SCHEDULED**, **PROCESSING**, **BOOKED**, **FAILED** and **CANCELLED**. External transfers balance against a system settlement account; every booked payment creates exactly one debit and one credit.

Architecture



Money travels through the API as decimal strings, uses **decimal** in Go and is stored as PostgreSQL **NUMERIC**. Ledger writes and balance changes run inside PostgreSQL transactions with stable account locking.

Technology

Layer	Stack
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS, Zustand
Backend	Go 1.26, Chi Router, sqlc, JWT, zerolog
Database	PostgreSQL 16, additive migrations, immutable ledger entries
Background work	In-process scheduler or standalone Go worker
Delivery	Docker Compose, GitHub Actions, CodeQL

Quick start

Docker Compose

Requirements: Docker Desktop with Docker Compose.

```
cp .env.example .env
# Replace JWT_SECRET with a unique value of at least 32 random characters.
```

```
docker compose up --build
```

Service	Local URL
Banking UI	localhost:3000
API health	localhost:8383/health
Swagger UI	localhost:8383/swagger/index.html
MailHog inbox	localhost:8425
Jaeger traces	localhost:16686
Grafana dashboards	localhost:3001
Prometheus targets	localhost:9090/targets
PostgreSQL	localhost:5433

Stop the stack with `docker compose down`. Add `-v` only when you intentionally want to remove the local PostgreSQL volume.

Grafana is provisioned with the `Prometheus` datasource and two RabbitMQ dashboards. The local default user is `admin`; set the independent `GRAFANA_ADMIN_PASSWORD` in `.env` before starting the stack. RabbitMQ's metrics port `15692` is intentionally internal-only: Prometheus scrapes it every 15 seconds.

Optional demo data

```
DEMO_SEED=true
DEMO_SEED_PASSWORD=<unique-secret-with-15-to-72-bytes>
```

The idempotent seed creates the fictional users `helga.mueller@pehlione.com`, `jonas.schneider@pehlione.com` and `sofia.wagner@pehlione.com`, demo accounts, beneficiaries and sample payments. All names, addresses, balances and transactions must remain fictional; never enter real personal data in a demo environment.

Hybrid local development (Windows)

The development launcher keeps PostgreSQL and MailHog in Docker, runs the Linux backend binary in a lightweight container, and runs Next.js natively with npm for fast refresh:

```
.\start.ps1
```

The equivalent backend build command is:

```
cd backend
```

```
mage -v build:linux
```

This produces the single executable `backend/bin/linux/ledger`. PostgreSQL and migrations are managed through `docker-compose.dev.yml`; the backend is exposed on `http://localhost:8383` by default and the frontend starts with `npm run dev`. Set `BACKEND_DEV_PORT` to override the backend host port. Stop the Compose services with `.\start.ps1 -Stop`.

Optional administrator bootstrap

```
ADMIN_SEED_EMAIL=admin@example.invalid
ADMIN_SEED_PASSWORD=<unique-secret-with-15-to-72-bytes>
```

Both values are required together. Do not commit real passwords. Public registrations always receive the `CUSTOMER` role and cannot promote themselves.

Development

- Backend commands
- Frontend commands

API at a glance

Purpose	Endpoints
Authentication	POST /register, POST /login, POST /forgot-password, POST /reset-password, POST /logout, GET /session
Customer profile	GET /profile, PATCH /profile
Accounts	GET /accounts, GET /accounts/{id}, GET /accounts/{id}/transactions
Payee verification	POST /payees/verify
Payments	POST /payments, GET /payments, POST /payments/{id}/confirm, POST /payments/{id}/cancel
Standing orders	POST /standing-orders, GET /standing-orders, PATCH /standing-orders/{id}, DELETE /standing-orders/{id}
Live updates	GET /events
Administration	/admin/* role-protected endpoints

`POST /payments` requires an `Idempotency-Key`. Reusing a key with the same normalized intent returns the existing order; reusing it with changed payment data returns a conflict. Payment creation alone never books funds—the client must complete VoP, display the summary and explicitly confirm the payment.

Notification service

Password-reset messages use an explicit, token-authenticated HTTP command. Committed payment activity is delivered asynchronously: Banking writes a versioned event to its PostgreSQL transactional outbox, then a publisher sends it to RabbitMQ; Notification consumes it and delivers through SMTP in local development or the Resend HTTPS API when configured. Notification never queries Banking tables; it owns only a processed-event ID store for consumer idempotency. Password-reset links expire after 15 minutes, are single-use and revoke all earlier sessions when consumed.

`./start.ps1` configures the backend to send locally through MailHog. Open localhost:8425 to inspect captured messages; MailHog never forwards them to the public internet.

```
NOTIFICATION_SERVICE_URL=http://localhost:8490
NOTIFICATION_SERVICE_TOKEN=<independent-random-secret>
RABBITMQ_URL=amqp://<user>:<url-encoded-password>@localhost:5672/
RABBITMQ_RETRY_DELAY=5s
RABBITMQ_MAX_RETRIES=3
RESEND_API_KEY=re_...
MAIL_FROM=Pehlione DemoBank <banking@pehlione.com>
FRONTEND_URL=http://localhost:3000
```

When `SMTP_HOST` is configured in Notification it takes precedence over Resend. The sender domain in `MAIL_FROM` must be verified in Resend with its SPF and DKIM records. Never commit provider credentials. RabbitMQ retry queues handle transient delivery failures, while malformed and exhausted events go to `notification.payment.dlq`. Financial commits never wait for RabbitMQ or the provider. Delivery is at least once—not exactly once; event IDs suppress normal duplicate broker deliveries, but a provider-accepted email immediately before a process crash may still be resent.

Security model

- Source-account ownership is enforced by the API.
- Account and payment lists return masked IBANs.
- A full IBAN is available only through an owner-authorized account detail request.
- Cookies use Secure, HttpOnly and SameSite protections in production.
- CSRF checks, CORS allowlists and rate limits protect sensitive endpoints.
- Strict request decoding reduces mass-assignment risk.
- Profile changes are owner-scoped and audited without copying address or contact data into audit metadata.
- Stable locking and transactional writes prevent partial ledger updates.
- Logs must never contain credentials, JWTs or unmasked personal data.

Read the repository's [security pentest report](#) for the implemented findings and mitigations.

For a control-by-control source file map and the remaining real-bank requirements, see [CYBER_SECURITY.md](#) or the [German version](#).

Project structure

```
├── backend/
```

		cmd/	API and worker entry points
		internal/identity/	Credential policy and authentication application
port		internal/account/	IBAN, ownership and profile application port
		internal/ledger/	Ledger application service, port and pure posting
domain		internal/payment/	Payment orchestration, pure lifecycle/intent
domain and scheduling		internal/notification/	Provider-neutral notification port
		internal/platform/	HTTP/database, outbox, RabbitMQ and Notification
adapters		internal/architecture/	Enforced module dependency rules
		postgres/	Migrations, queries and sqlc output
		frontend/	
		app/	Next.js routes
		components/banking/	Banking UI and transfer wizard
		lib/	API client, state and types
		docker-compose.yml	
		docker-compose.dev.yml	

Reference material

- [European Central Bank — Instant payments and Verification of Payee](#)
- [European Payments Council — SEPA Credit Transfer](#)
- [Deutsche Bundesbank — SEPA guidance](#)

Author and license

Created and maintained by **Mustafa Özdemir**. Licensed under the [MIT License](#).

This repository is for education and simulation only. It is not legal, compliance, financial or operational banking advice.